Data Mining Final Project Sprint 1 Documentation and Summary

Graduate Project Lead : Kiefer Armand
Undergraduate Modelers: Keahi Temple, William Becker, Reese Haydel
2/8/2026

Summary:

This week, our group selected the **Command and Control** system and identified potential subsystem improvements, including alert thresholds, reporting cycles, risk judgment, and reporting processes. We also selected our dataset (**UAVSwarm**) and began working with the data. Undergraduate modelers completed the tasks of data understanding, data quality assessment, and data exploration. The graduate project lead reviewed the modelers' work and incorporated new project guidance provided by the professor to organize the project and plan future sprints.

Using the CRISP-DM cycle, the project focus was refined to **situation assessment**. There were no major blockers this week. All documentation produced for individual assignments has been compiled below. Next week, we will complete preprocessing and begin applying modeling techniques to the data.

Project outline - Kiefer Armand

Prior:
Traditional C2 systems rely on predefined rules and static thresholds to assess operational situations, which limits adaptability to changing environments and emerging behaviors.

New:
Replace static, rule-based situation assessment with a data-driven assessment function that learns normal operational patterns and identifies deviations dynamically.

Affected Views:
OV-5: *Assess Situation*

SV-4: *Information Processing*

SV-10: *Rules governing behavior*

Map dataset:
Bounding boxes – detected entities

Frame index – time

Object ID – Asset

Confidence score – Sensor Reliability

Trajectories – Behavioral history

Planned/proposed Modeling:

Unsupervised Learning

- Clustering
- Density estimations

Command and control element: We want a command-and-control system to learn what normal actually looks like, rather than what it “Should Be”.

Anomaly Detection:

- Isolation Forests

- LOF

Command and control element: We want a command-and-control system to learn what normal looks like, rather than what it “Should Be”.

Plan of Action:

Data Understanding: (Pretty much done, make some minor tweaks to the dataset)

Temporal aggregation/ Feature Engineering: command and control doesn’t look at data per sample, but rather situations. We don’t exactly have that, so we are going to make them. We will be compressing the dataset into frame windows to create situations these situations. First, we group the data by the frames, then aggregate the data into frame windows (size at our discretion but probably 3-5 frame Id’s per window). That way we compress the data of however many rows into a single row that represents a situation rather than a point. The reason we are doing this is because of how Command and Control works. It doesn’t operate at a sensor level resolution situation assessment but rather acts as an abstraction layer. Think about it like this: instead of controlling a single unit that is constantly giving you feedback and information, you must control the entire army. I know this may seem a bit confusing, don’t hesitate to ask me if you are confused.

Normalize Data: probably Z-score

Unsupervised Learning: we will probably try multiple different approaches to see which fits best: DBSCAN, K_Means, Isolation forest

Cluster interpretation Examples (This will change based on we our outputs/results are):

- State A: Low activity / dispersed
- State B: High density / high confidence
- State C: Transitional
- Outliers: Anomalous situations

Replace the static rules with the learned normality:

Previous: Alert when count > N

New: alert when situation deviates significantly from the learned cluster baseline

Validation/Security: Mostly on me to verify all of this, y’all will more than likely be working on other things during this time like wrap up, presentation materials, the report, and our DoDaf Mappings

DoDAF Mappings: Link each element of the project to the different DoDaf views

Presentation and wrap up: pretty much anything we haven’t finished yet

Data Understanding - Keahi Temple

Identify the Source:

The dataset is hosted on GitHub at https://github.com/UAVSwarm/UAVSwarm-dataset, and was created by researchers to support UAV swarm detection and tracking research. This dataset consists of manually collected and annotated images, and is meant to help train and evaluate various models that can detect drones in these image frames.

Load/inspect the Data:

The UAVSwarm dataset is organized into multiple directories that separate raw image data, annotations, detection outputs, and dataset splits. The top-level structure consists of an annotations directory and separate train and test directories. The train and test directories contain multiple numbered UAVSwarm sequence folders. Each sequence represents a time-ordered series of image frames capturing UAV swarm activity under a specific scene or condition. Within these folders, the image files are **jpg** format, while the annotation and detection files are **txt** format. Within the annotations folder, there are 4 **json** files, titled test, train, train_half, and val_half. There are also **ini** files in each swarm folder that contain sequence level metadata.

A single image frame represents one sample. The dataset contains **12,598 annotated image frames** distributed across multiple UAV swarm sequences. Each image file can contain multiple UAV instances, with each instance being a separate row in the annotation/detection files. Each one of those files contains **9 features**.

No merging should be necessary, but logical association between image frames and their detection/annotation files will be.

List and Describe the Features:

The features are Frame index, object ID, bounding box coordinates (x, y, width, height), confidence/validity flag, object class, visibility/ignored fields. To

describe the features, I have created a table below:

Feature #	**Feature Name**	**Data Type**	**Description**
1	Frame Number	Integer	Index of the video frame in which the UAV appears
2	Object ID	Integer	Unique identifier for a UAV across frames (−1 in detection files)
3	Bounding Box X	Integer	X-coordinate of the upper-left corner of the bounding box
4	Bounding Box Y	Integer	Y-coordinate of the upper-left corner of the bounding box
5	Bounding Box Width	Integer	Width of the bounding box in pixels
6	Bounding Box Height	Integer	Height of the bounding box in pixels
7	Confidence / Validity Flag	Integer/Float	Confidence score or activity indicator
8	Object Class	Integer	Category label (always 1 for UAVs)

9	Visibility Ratio	Float	Visibility of the UAV, ranging from 0 to 1

Detection (det) files represent UAV locations **predicted** by YOLOX detectors, and include the frame index, object ID, bounding box coordinates (x, y, width, height), confidence score, and ignored fields. The ignored fields are all set to -1 and are **not used** during detection evaluation.

The **annotation** files, labeled gt (ground truth) and represent **manually annotated** UAV trajectories. These files include the same first 6 features as detection files, but feature 7 (confidence score) is also a **validity flag**, which indicates whether the UAV instance is active and should be included in evaluation. The **Object class** is set to **1**, as the dataset only contains a single UAV category.The ninth feature represents the visibility ratio of each UAV, accounting for occlusion and image boundary clipping.

Assess Relevance to System Problem

The primary system problem that the UAVSwarm dataset addresses is the detection and tracking of many UAV swarms **simultaneously**. From a data mining perspective, the goal will be to identify which features actually contribute meaningful information for patterns in UAV behavior.

In the table below, each feature is evaluated based on usefulness for UAV detection, multi-object tracking, and swarm behavior analysis.

Feature	**Relevant?**	**Justification**
Frame Number	Yes	Provides temporal ordering required for tracking UAV trajectories across frames
Object ID (gt)	Yes	Enables identity persistence and trajectory reconstruction

Object ID (det)	No	Always set to −1 and provides no modeling value
Bounding Box	Yes	Core spatial features required for detection and tracking
Confidence Score (det)	Yes	Useful for filtering low-confidence detections and improving tracking robustness
Validity Flag (gt)	Yes	Determines whether an annotated UAV should be included in evaluation
Object Class	Limited	Always set to 1; provides no discriminative value for classification
Visibility Ratio	Yes	Useful for handling occlusion and evaluating tracking reliability
Ignored Fields (det)	No	Always set to −1 and not used in evaluation

The most useful features are bounding box coordinates, frame number, object ID (gt only), confidence score, and visibility ratio. Object class is **conditionally** useful. **Ignored fields** and **detection object IDs** are **irrelevant** and will not be important for modeling.

DATA Quality - William Becker

1 Check for data completeness:

1. Identify any missing values.
    a. There are no missing values in the dataset.

```python
missing_count = df.isnull().sum()
missing_percent = df.isnull().mean() * 100

missing_summary = pd.DataFrame({
    "Missing Count": missing_count,
    "Missing %": missing_percent
}).sort_values(by="Missing %", ascending=False)

missing_summary
```

	Missing Count	Missing %
1	0	0.0
-1	0	0.0
117	0	0.0
145	0	0.0
31	0	0.0
25	0	0.0
1.1	0	0.0
-1.1	0	0.0
-1.2	0	0.0

2. Flag features/samples with excessive missing data
    a. None of the features have excessive missing values.

```python
cols_to_drop = missing_percent[missing_percent > 30].index
cols_to_drop
```

```
Index([], dtype='object')
```

2. Handle missing values (use appropriate method, remember the class activity on missing values)

1. Drop missing values
    a. Because this dataset has no missing values, this part did nothing. 2. Replace missing values (imputation)
    a. See 2.1.a.

3. Check data Validity
    1. Confirm numeric fields fall within logical bounds

```
df[numeric_cols].describe()
(df[numeric_cols] < 0).sum()
```

```
...
            0
1           0
-1      49660
117        49
145       268
31          0
25          0
1.1         0
-1.1    49660
-1.2    49660
-1.3    49660

dtype: int64
```

2. Verify categorical fields use allowed values

```
for col in categorical_cols:
    print(f"\n{col}")
    print(df[col].unique())
```

```
...
UAVSwarm-09
['uavswarm-09' 'uavswarm-57' 'uavswarm-11' 'uavswarm-05' 'uavswarm-17'
 'uavswarm-31' 'uavswarm-07' 'uavswarm-59' 'uavswarm-03' 'uavswarm-71'
 'uavswarm-47' 'uavswarm-67' 'uavswarm-25' 'uavswarm-37' 'uavswarm-69'
 'uavswarm-51' 'uavswarm-35' 'uavswarm-13' 'uavswarm-49' 'uavswarm-43'
 'uavswarm-15' 'uavswarm-39' 'uavswarm-33' 'uavswarm-41' 'uavswarm-45'
 'uavswarm-55' 'uavswarm-23' 'uavswarm-29' 'uavswarm-01' 'uavswarm-53'
 'uavswarm-19' 'uavswarm-65' 'uavswarm-27' 'uavswarm-21' 'uavswarm-63'
 'uavswarm-61']
```

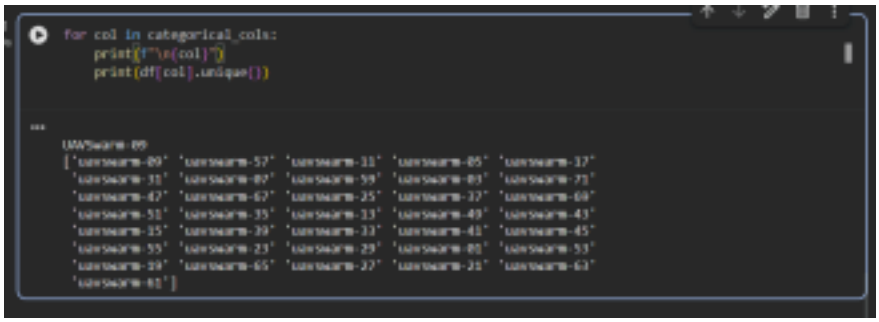
3. Short description of what each field represents

Feature #	Feature Name	Data Type	Description
1	Frame Number	Integer	Index of the video frame in which the UAV appears
2	Object ID	Integer	Unique identifier for a UAV across frames (−1 in detection files)
3	Bounding Box X	Integer	X-coordinate of the upper-left corner of the bounding box
4	Bounding Box Y	Integer	Y-coordinate of the upper-left corner of the bounding box
5	Bounding Box Width	Integer	Width of the bounding box in pixels

5	Bounding Box Width	Integer	Width of the bounding box in pixels
6	Bounding Box Height	Integer	Height of the bounding box in pixels
7	Confidence / Validity Flag	Integer/Float	Confidence score or activity indicator
8	Object Class	Integer	Category label (always 1 for UAVs)
9	Visibility Ratio	Float	Visibility of the UAV, ranging from 0 to 1

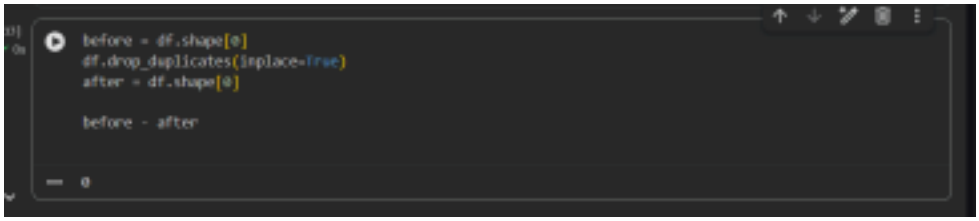
4 Data Consistency

1. Identify inconsistent labels or naming schemes

a. 

2. Check for duplicate samples

a. There appear to be no duplicate samples, as shown by this image.



5 Outlier detection:

1. Identify extreme values (use more than 1 method before labeling something as an outlier) 2. Z-Score, IQR, regression forest

a. Z-score:

```
z_scores = np.abs(stats.zscore(df[numeric_cols]))
z_outliers = (z_scores > 3).any(axis=1)
z_outliers.sum()


/tmp/ipython-input-2877647485.py:1: RuntimeWarning: Precision loss occurred in moment calculation due to
  z_scores = np.abs(stats.zscore(df[numeric_cols]))
np.int64(3407)
```

b. IQR Method:

```
Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

iqr_outliers = ((df[numeric_cols] < (Q1 - 1.5 * IQR)) |
                (df[numeric_cols] > (Q3 + 1.5 * IQR))).any(axis=1)

iqr_outliers.sum()


np.int64(9813)
```

6 Handle outliers (you may have to do a bit of research for this)

1. determine whether to drop or keep any value that is flagged as an outlier(there really no wrong answer here just make sure you document your justification for dropping or keeping.
    a. I dropped the values that were marked as outliers by both the z-score and IQR method because it was a relatively small portion of the dataset (about 3,000), so it made sense to get rid of these outliers.

```
df = df[~(z_outliers & iqr_outliers)]
df.shape


(46253, 11)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 46253 entries, 0 to 49659
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   1            46253 non-null  int64
 1   -1           46253 non-null  int64
 2   117          46253 non-null  int64
 3   145          46253 non-null  int64
 4   31           46253 non-null  int64
 5   25           46253 non-null  int64
 6   1.1          46253 non-null  int64
 7   -1.1         46253 non-null  int64
 8   -1.2         46253 non-null  int64
 9   -1.3         46253 non-null  int64
 10  UAVSwarm-09  46253 non-null  object
dtypes: int64(10), object(1)
memory usage: 4.2+ MB
```

	1	-1	117	145	31	25	1.1	-1.1	-1.2
count	46253.000000	46253.0	46253.000000	46253.000000	46253.000000	46253.000000	46253.0	46253.0	46253.0
mean	99.803775	-1.0	448.798110	241.833524	25.906838	19.424794	1.0	-1.0	-1.0
std	75.325876	0.0	283.296393	147.365059	15.330898	8.220613	0.0	0.0	0.0
min	1.000000	-1.0	-52.000000	-31.000000	10.000000	10.000000	1.0	-1.0	-1.0
25%	43.000000	-1.0	230.000000	145.000000	17.000000	14.000000	1.0	-1.0	-1.0
50%	85.000000	-1.0	391.000000	219.000000	21.000000	17.000000	1.0	-1.0	-1.0
75%	137.000000	-1.0	610.000000	312.000000	29.000000	21.000000	1.0	-1.0	-1.0
max	403.000000	-1.0	1517.000000	827.000000	108.000000	65.000000	1.0	-1.0	-1.0

Data Exploration - Reese Haydel

Loading the Cleaned Data and Performing a Structure Scan:

```
import pandas as pd
import numpy as np

# Upload in Colab: from google.colab import files; files.upload()
df = pd.read_csv("combined_cleaned.csv")

print("Shape:", df.shape)
display(df.head())
print("\nDtypes:")
print(df.dtypes)
```

Shape: (44493, 20)

Feature Name	Data Type
1	int64
-1	int64
117	int64
145	int64
31	int64
25	int64
1.1	int64
-1.1	int64
-1.2	int64
-1.3	int64
1.2	int64
1.3	int64
117.1	int64
145.1	int64

31.1	int64
25.1	int64
1.4	int64
1.5	int64
1.6	int64
UAVSwarm 09	object

dtype: object

Verifying Data Quality:

```
# Missing values
missing = df.isna().sum().sort_values(ascending=False)
missing_pct = (missing / len(df) * 100).round(2)
quality = pd.DataFrame({"missing": missing, "missing_%": missing_pct})
display(quality[quality["missing"] > 0])

# Duplicates
dup_count = df.duplicated().sum()
print("Duplicate rows:", dup_count)

# Basic numeric sanity
num_cols = df.select_dtypes(include="number").columns
display(df[num_cols].describe().T)
```

Duplicate rows: 0

Target / Label Exploration:

```python
target_col = "UAVSwarm-09"

print("Unique classes:", df[target_col].nunique())
class_counts = df[target_col].value_counts()
display(class_counts)

# Class balance ratio
print("Largest/Smallest class ratio:",
      (class_counts.max() / class_counts.min())
      if class_counts.min() > 0 else "undefined")
```

Class Label	Count
UAVSwarm 61	3,543
UAVSwarm 11	3,356
UAVSwarm 25	3,336
UAVSwarm 01	2,984
UAVSwarm 13	2,499
UAVSwarm 49	2,145
UAVSwarm 45	1,967
UAVSwarm 41	1,601
UAVSwarm 05	1,530
UAVSwarm 47	1,423
UAVSwarm 07	1,394

UAVSw arm 03	1,354
UAVSw arm 51	1,336
UAVSw arm 43	1,310

UAVSwarm

55 1,294

UAVSw arm 15	1,286
UAVSw arm 37	1,251
UAVSw arm 19	1,217
UAVSw arm 29	1,151
UAVSw arm 63	1,080
UAVSw arm 69	913
UAVSw arm 71	827
UAVSw arm 27	772
UAVSw arm 33	733
UAVSw arm 17	669
UAVSw arm 53	663
UAVSw arm 39	519
UAVSw	494

arm 23	
UAVSw arm 67	340
UAVSw arm 59	322
UAVSw arm 31	321
UAVSw arm 09	271
UAVSw arm 57	261
UAVSw arm 21	206
UAVSw arm 65	125

Metric	Value
Total Classes	35

Largest Class Size 3,543

Smallest Class Size	125
Largest / Smallest Ratio	**28.344 : 1**

Scanning for Distribution and Outliers:

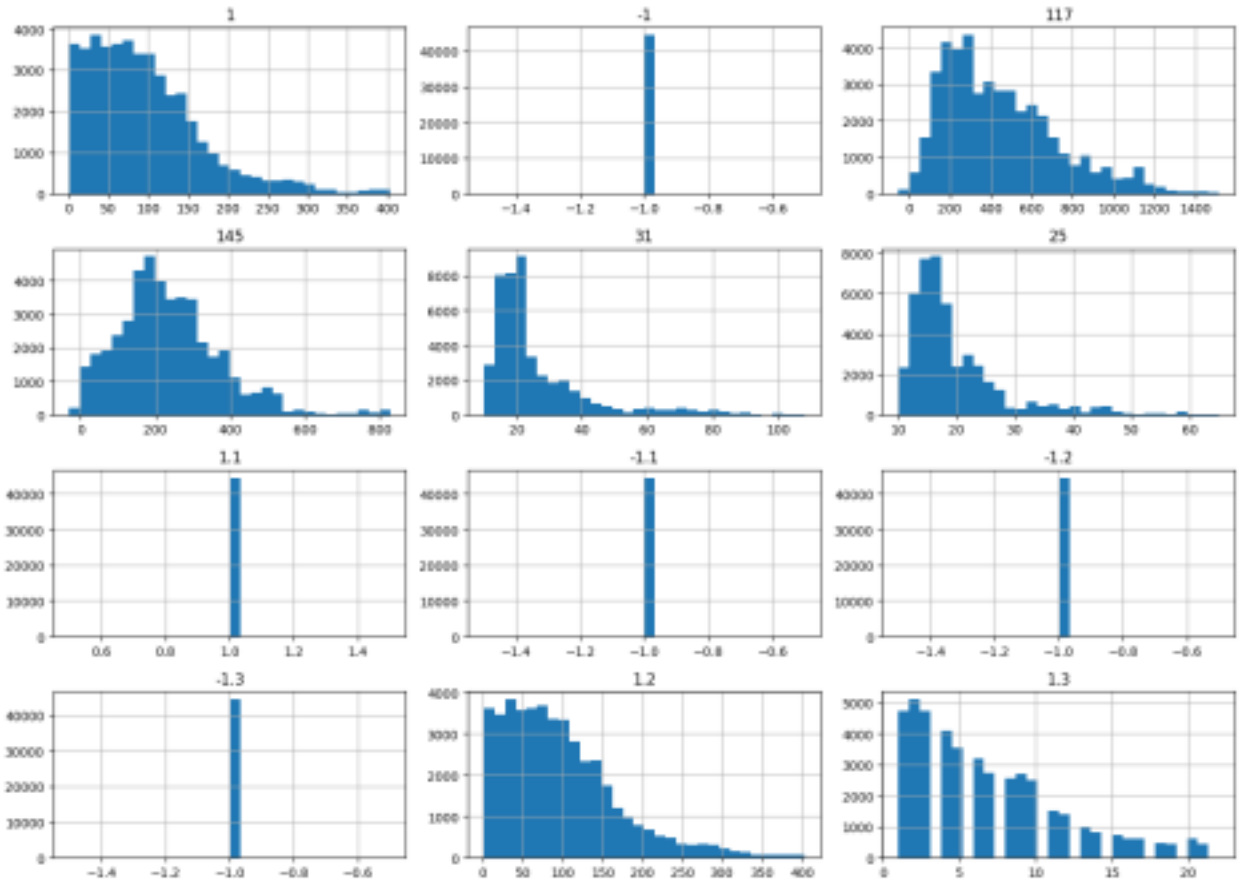
```python
import matplotlib.pyplot as plt

# Histograms for first 12 numeric columns (edit as needed)
cols = list(num_cols[:12])

df[cols].hist(bins=30, figsize=(14, 10))
plt.tight_layout()
plt.show()

# Simple outlier flag using IQR (not perfect, but quick)
outlier_summary = []
for c in num_cols:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
    frac = ((df[c] < lo) | (df[c] > hi)).mean()
    outlier_summary.append((c, float(frac)))

outlier_df = pd.DataFrame(outlier_summary,
columns=["feature","outlier_frac"]).sort_values("outlier_frac",
ascending=False)
display(outlier_df.head(15))
```



Relationships and/or Redundancy:

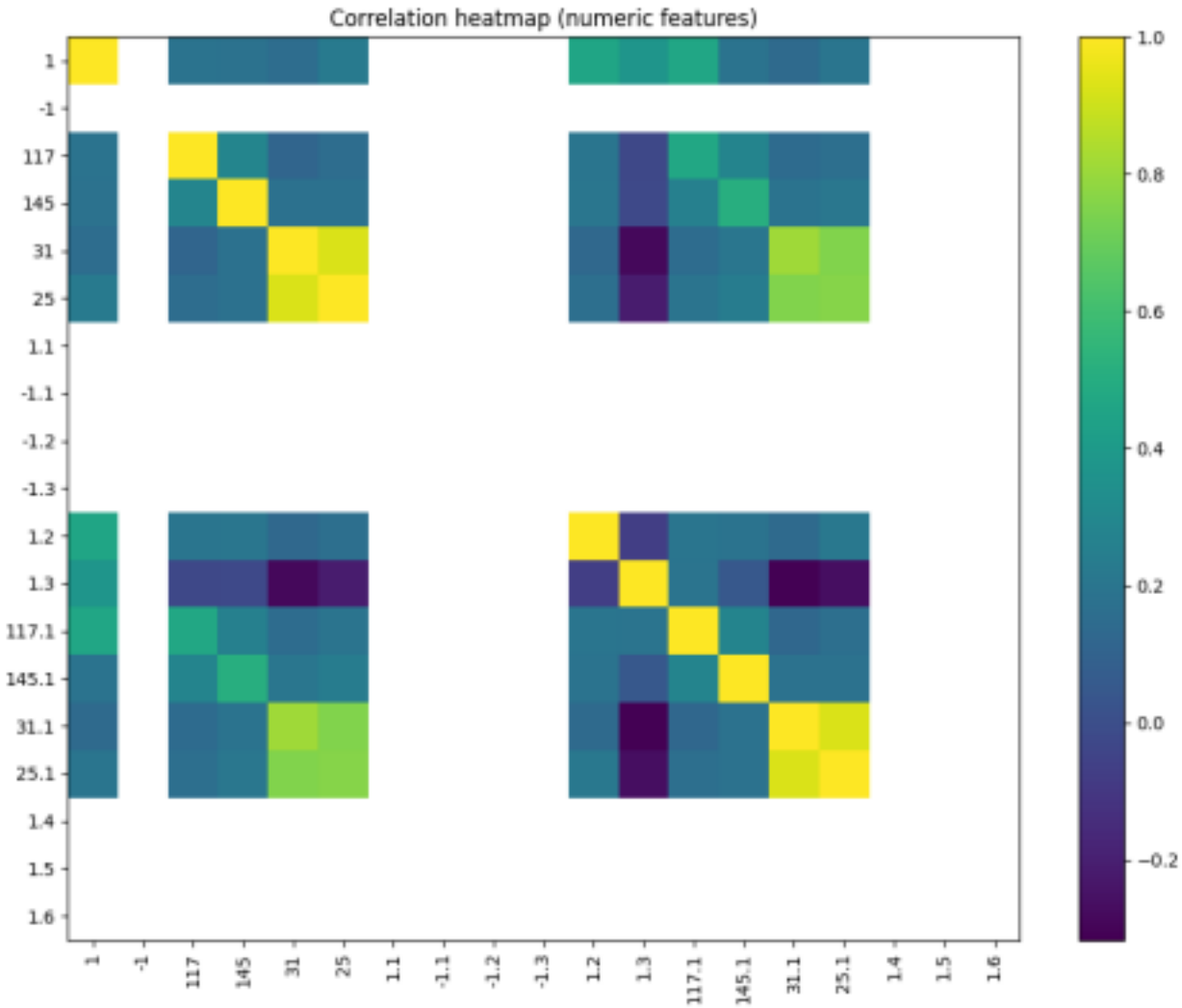
```python
import numpy as np

corr = df[num_cols].corr(numeric_only=True)

# Find highly correlated pairs (potential redundancy)
threshold = 0.95
pairs = []
cols = corr.columns
for i in range(len(cols)):
    for j in range(i+1, len(cols)):
        val = corr.iloc[i, j]
        if abs(val) >= threshold:
            pairs.append((cols[i], cols[j], float(val)))

pairs_df = pd.DataFrame(pairs,
columns=["feat_1","feat_2","corr"]).sort_values("corr", key=np.abs,
ascending=False)
display(pairs_df.head(30))

# Correlation heatmap (simple)
plt.figure(figsize=(10, 8))
plt.imshow(corr.values, aspect="auto")
plt.colorbar()
plt.title("Correlation heatmap (numeric features)")
plt.xticks(range(len(cols)), cols, rotation=90)
plt.yticks(range(len(cols)), cols)
plt.tight_layout()
plt.show()
```



Baseline Model for a Post Cleaning Sanity Check:

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

X = df[num_cols].copy()
y = df[target_col].copy()

# Encode labels if string
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=2000))
])

model.fit(X_train, y_train)
pred = model.predict(X_test)
print(classification_report(y_test, pred))
```

Class	Precision	Recall	F1-Score	Support
0	0.71	0.75	0.73	597
1	0.71	0.66	0.68	271
2	0.82	0.88	0.85	306
3	0.68	0.74	0.71	279
4	0.6	0.76	0.67	54
5	0.68	0.81	0.74	671
6	0.9	0.87	0.88	500
7	0.78	0.77	0.78	257
8	0.51	0.34	0.41	134

9	0.92	0.95	0.93	243
10	0.87	0.32	0.46	41
11	0.49	0.45	0.47	99
12	0.76	0.71	0.73	667

13 0.97 0.94 0.95 154

14	0.77	0.84	0.8	230
15	0.87	0.7	0.78	64
16	0.78	0.86	0.82	147
17	0.86	0.86	0.86	250
18	0.99	1	1	104
19	0.95	0.94	0.94	320
20	0.99	0.98	0.99	262
21	0.81	0.87	0.84	394
22	0.94	0.94	0.94	285
23	0.77	0.93	0.84	429
24	0.6	0.52	0.56	267
25	0.61	0.32	0.42	133
26	0.44	0.39	0.41	259
27	0.77	0.46	0.58	52
28	0.4	0.16	0.22	64
29	0.95	0.96	0.96	709
30	0.81	0.85	0.83	216
31	0.89	1	0.94	25
32	0.56	0.46	0.5	68
33	0.91	0.91	0.91	183
34	0.96	0.87	0.91	165

Accuracy			0.79	8899
Macro Avg	0.77	0.74	0.74	8899
Weighted Avg	0.79	0.79	0.79	8899